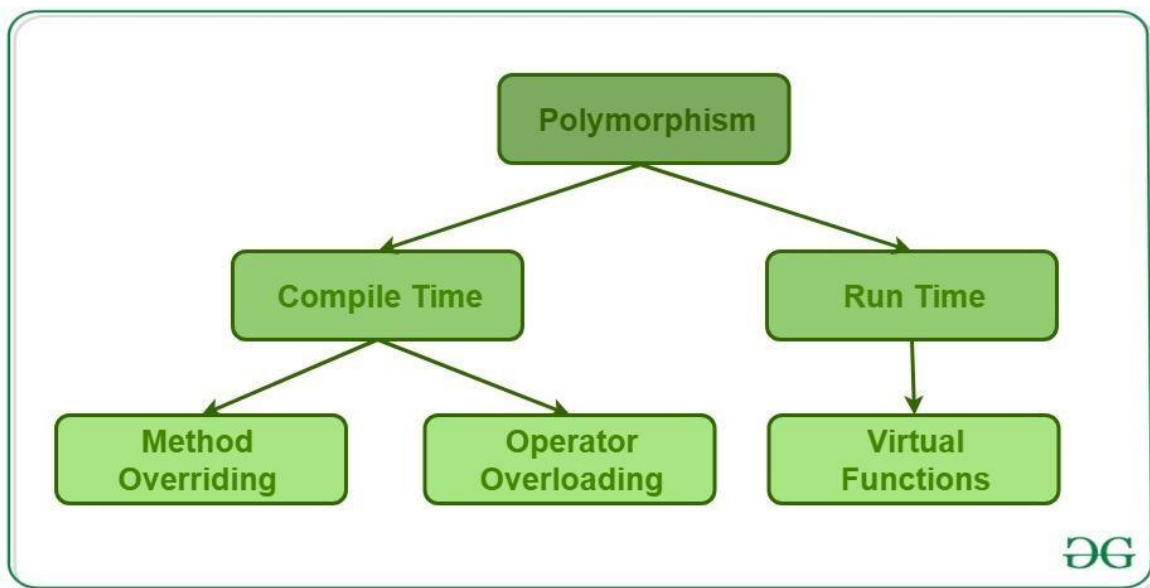


Polymorphism in C++

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. Real life example of polymorphism, a person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism. Polymorphism is considered as one of the important features of Object Oriented Programming.

In C++ polymorphism is mainly divided into two types:

- Compile time Polymorphism
- Runtime Polymorphism



1. **Compile time polymorphism:** This type of polymorphism is achieved by function overloading or operator overloading.

- **Function Overloading:** When there are multiple functions with same name but different parameters then these functions are said to be **overloaded**. Functions can be overloaded by **change in number of arguments** or/and **change in type of arguments**.

```
// C++ program for function overloading
#include <bits/stdc++.h>

using namespace std;
class Geeks
{
public:

    // function with 1 int parameter
    void func(int x)
    {
        cout << "value of x is " << x << endl;
    }

    // function with same name but 1 double parameter
    void func(double x)
```

```

    {
        cout << "value of x is " << x << endl;
    }

    // function with same name and 2 int parameters
    void func(int x, int y)
    {
        cout << "value of x and y is " << x << ", " << y << endl;
    }
};

int main() {

    Geeks obj1;

    // Which function is called will depend on the parameters passed
    // The first 'func' is called
    obj1.func(7);

    // The second 'func' is called
    obj1.func(9.132);

    // The third 'func' is called
    obj1.func(85,64);    return
0;
}

```

Output:

```

value of x is 7 value of x
is 9.132 value of x and y
is 85, 64

```

In the above example, a single function named *func* acts differently in three different situations which is the property of polymorphism.

- **Operator Overloading:** C++ also provide option to overload operators. For example, we can make the operator ('+') for string class to concatenate two strings. We know that this is the addition operator whose task is to add two operands. So a single operator '+' when placed between integer operands , adds them and when placed between string operands, concatenates them. **Example:**

```

// CPP program to illustrate
// Operator Overloading
#include<iostream>
using namespace std;

class Complex { private:
    int real, imag; public:
    Complex(int r = 0, int i =0) {real = r;  imag = i;}

    // This is automatically called when '+' is used with
    // between two Complex objects
    Complex operator + (Complex const &obj) {

```

```

        Complex res;    res.real =
    real + obj.real;    res.imag =
    imag + obj.imag;    return res;
    }
    void print() { cout << real << " + i" << imag << endl; }
};

```

```

int main()
{
    Complex c1(10, 5), c2(2, 4);
    Complex c3 = c1 + c2; // An example call to "operator+"
    c3.print();
}

```

Output:

```
12 + i9
```

In the above example the operator '+' is overloaded. The operator '+' is an addition operator and can add two numbers(integers or floating point) but here the operator is made to perform addition of two imaginary or complex numbers. To learn operator overloading in details visit [this link](#).

2. **Runtime polymorphism:** This type of polymorphism is achieved by Function Overriding.
- **Function overriding** on the other hand occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be **overridden**.

// C++ program for function overriding

```

#include <bits/stdc++.h> using
namespace std;

```

```

class base
{ public:
    virtual void print ()
    { cout<< "print base class" <<endl; }

    void show ()
    { cout<< "show base class" <<endl; }
};

```

```

class derived:public base
{ public:
    void print () //print () is already virtual function in derived class,
    //we could also declared as virtual void print () explicitly
    { cout<< "print derived class" <<endl; }

    void show ()
    { cout<< "show derived class" <<endl; }
};

```

```

//main function
int main()
{

```

```

base *bptr;
derived d;
bptr = &d;

//virtual function, binded at runtime (Runtime polymorphism) bptr-
>print();

// Non-virtual function, binded at compile time bptr-
>show();

return 0;
}
Output:

```

```

print derived class show
base class

```

Virtual Function in C++

A virtual function is a member function which is declared within a base class and is redefined(Overriden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.

- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for function call.
- They are mainly used to achieve **Runtime polymorphism** • Functions are declared with a **virtual** keyword in base class.
- The resolving of function call is done at Run-time.

Rules for Virtual Functions

1. Virtual functions cannot be static and also cannot be a friend function of another class.
2. Virtual functions should be accessed using pointer or reference of base class type to achieve run time polymorphism.
3. The prototype of virtual functions should be same in base as well as derived class.
4. They are always defined in base class and overridden in derived class. It is not mandatory for derived class to override (or re-define the virtual function), in that case base class version of function is used.
5. A class may have **virtual destructor** but it cannot have a virtual constructor.

Compile-time(early binding) VS run-time(late binding) behavior of Virtual Functions

Consider the following simple program showing run-time behavior of virtual functions.

```

// CPP program to illustrate
// concept of Virtual Functions

```

```

#include <iostream> using
namespace std;

```

```

class base { public:
    virtual void print()
    {
        cout << "print base class" << endl;
    }

    void show()
    {
        cout << "show base class" << endl;
    }
};

```

```

class derived : public base {
public:    void print()
    {
        cout << "print derived class" << endl;
    }

    void show()
    {
        cout << "show derived class" << endl;
    }
};

```

```

int main()
{
    base* bptr;
    derived d;
    bptr = &d;

    // virtual function, binded at runtime
    bptr->print();
    // Non-virtual function, binded at compile time
    bptr->show();
}

```

Output:

```

print derived class
show base class

```

Explanation: Runtime polymorphism is achieved only through a pointer (or reference) of base class type. Also, a base class pointer can point to the objects of base class as well as to the objects of derived class. In above code, base class pointer 'bptr' contains the address of object 'd' of derived class.

Late binding(Runtime) is done in accordance with the content of pointer (i.e. location pointed to by pointer) and Early binding(Compile time) is done according to the type of pointer, since print() function is declared with virtual keyword so it will be bound at run-time (output is *print derived class* as pointer is pointing to object of derived class) and show() is nonvirtual so it will be bound during compile time(output is *show base class* as pointer is of base type).

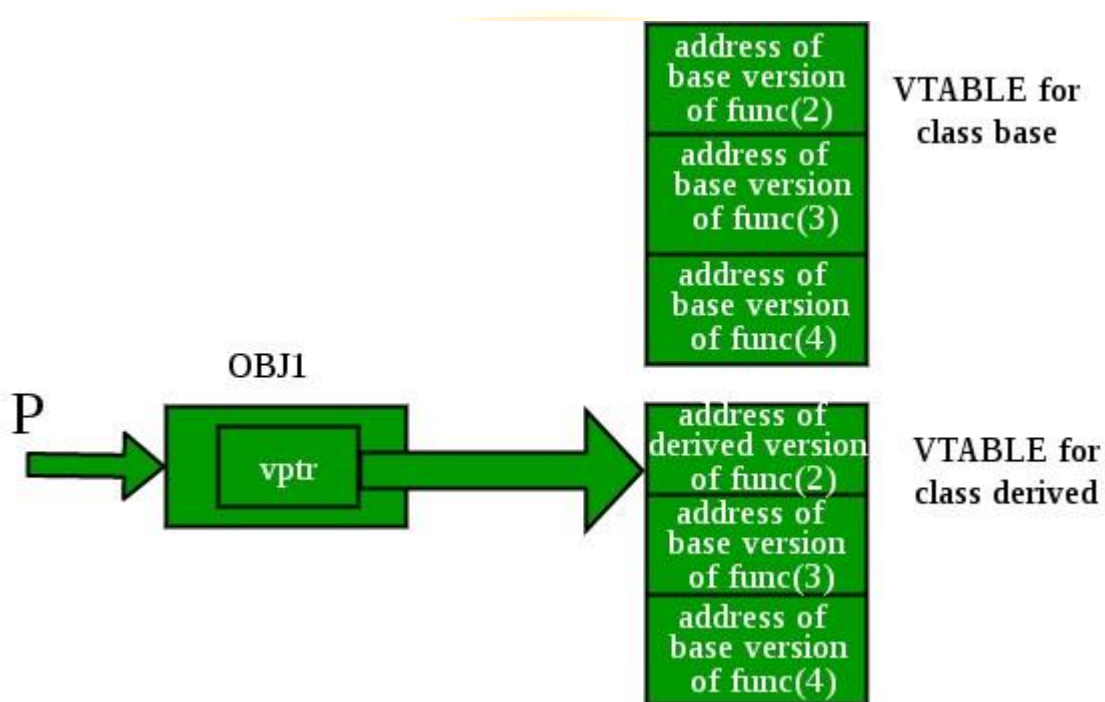
NOTE: If we have created a virtual function in the base class and it is being overridden in the derived class then we don't need virtual keyword in the derived class, functions are automatically considered as virtual functions in the derived class.

Working of virtual functions(concept of VTABLE and VPTR)

As discussed [here](#), If a class contains a virtual function then compiler itself does two things:

1. If object of that class is created then a **virtual pointer(VPTR)** is inserted as a data member of the class to point to VTABLE of that class. For each new object created, a new virtual pointer is inserted as a data member of that class.
2. Irrespective of object is created or not, a **static array of function pointer called VTABLE** where each cell contains the address of each virtual function contained in that class.

Consider the example below:



```
// CPP program to illustrate
// working of Virtual Functions
#include <iostream> using
namespace std;
```

```
class base {
public:
    void fun_1() { cout << "base-1\n"; }
    virtual void fun_2() { cout << "base-2\n"; }
    virtual void fun_3() { cout << "base-3\n"; }
    virtual void fun_4() { cout << "base-4\n"; }
};
```

```
class derived : public base { public:    void
fun_1() { cout << "derived-1\n"; }    void
fun_2() { cout << "derived-2\n"; }    void
fun_4(int x) { cout << "derived-4\n"; }
};
```

```
int main()
```

```

{   base* p;
derived obj1;
p = &obj1;

    // Early binding because fun1() is non-virtual
    // in base    p-
>fun_1();

    // Late binding (RTP)    p-
>fun_2();

    // Late binding (RTP)    p-
>fun_3();

    // Late binding (RTP)    p-
>fun_4();

    // Early binding but this function call is
    // illegal(produces error) because pointer
    // is of base type and function is of
    // derived class
// p->fun_4(5);
}
Output:

```

```

base-1
derived-2
base-3 base-4

```

Explanation: Initially, we create a pointer of type base class and initialize it with the address of the derived class object. When we create an object of the derived class, the compiler creates a pointer as a data member of the class containing the address of VTABLE of the derived class. Similar concept of **Late and Early Binding** is used as in above example. For fun_1() function call, base class version of function is called, fun_2() is overridden in derived class so derived class version is called, fun_3() is not overridden in derived class and is virtual function so base class version is called, similarly fun_4() is not overridden so base class version is called.

NOTE: fun_4(int) in derived class is different from virtual function fun_4() in base class as prototype of both the function is different.

A pure virtual function (or abstract function) in C++ is a **virtual function** for which we don't have implementation, we only declare it. A pure virtual function is declared by assigning 0 in declaration. See the following example.

```

// An abstract class
class
Test
{
    // Data members of class public:
    // Pure Virtual Function
    virtual void show() = 0;

    /* Other members */
};

```

A complete example:

A pure virtual function is implemented by classes which are derived from a Abstract class. Following is a simple example to demonstrate the same.

```
#include<iostream> using  
namespace std;
```

```
class Base  
{   int  
x;  
public:  
    virtual void fun() = 0;  
int getX() { return x; }  
};
```

```
// This class inherits from Base and implements fun() class  
Derived: public Base  
{   int  
y;  
public:  
    void fun() { cout << "fun() called"; }  
};
```

```
int main(void)  
{  
    Derived d;  
d.fun();  
return 0;  
}
```

Output:

```
fun() called
```